

KaraOK Security Architecture and Design

This document provides a comprehensive overview of the security architecture, mechanisms, and design decisions implemented in the KaraOK codebase. It serves as a technical reference detailing where and how security requirements are met across the Flutter frontend application and the Flask/MySQL backend service.

Table of Contents

1. UI Security
 2. System Security, Integrity, Redundancies, and Errors
 3. Authentication and Authorization
 4. Password Hashing
 5. Session Management
 6. Role-Based Access Control (RBAC)
 7. Input Validation and Sanitization
 8. Forgot Password Flow
 9. Audit Logging
 10. Security Verification and Regression Tests
-

1. UI Security

The Flutter frontend application manages user interactions securely by enforcing boundary checks, obfuscating sensitive inputs, and controlling user navigation according to authentication states.

Implementation Details

- **Secure Token Storage:** The application leverages the platform's secure storage hardware (Keychain on iOS, Keystore on Android) using the `flutter_secure_storage` package in [api_service.dart](#). Access and refresh tokens are never written to plaintext shared preferences or local SQLite databases.
- **Obfuscated Inputs:** Form fields for passwords and sensitive credentials in [login_screen.dart](#), [signup_screen.dart](#), and [change_password_screen.dart](#) implement `obscureText: true` by default. Obfuscation can be toggled using standard eye-icon buttons.
- **Mandatory Password Change Routing:** If a user logs in with a temporary password (after a reset request), the API returns `requires_password_change: true`. In [login_screen.dart](#), the frontend intercepts this flag and immediately redirects the user to the `ChangePasswordScreen(forceChange: true)`, clearing the navigation stack so the user cannot bypass the screen to access the application.
- **Guest Assessment Limits:** To prevent guest assessment abuse locally, [guest_assessment_service.dart](#) writes a key `karaok_guest_assessment_used_v1` to `FlutterSecureStorage` upon a successful

guest check. Subsequent attempts check this flag in [splash_screen.dart](#) to block excessive guest usage on the device.

- **HTTP Response Headers:** The backend adds browser security headers on every response within [app.py](#) via the `@app.after_request` hook:

- **Cache-Control:** `no-store` (prevents caching of authenticated pages/data)
- **X-Frame-Options:** `DENY` (anti-clickjacking)
- **X-Content-Type-Options:** `nosniff` (prevents MIME sniffing)
- **Referrer-Policy:** `no-referrer` (stops metadata leaks via referrer headers)

2. System Security, Integrity, Redundancies, and Errors

General application integrity, data safety, and server error handling are designed to defend against common attack vectors and information disclosures.

Implementation Details

- **Error Masking:** Database queries and system execution errors are caught centrally in [app.py](#). A backend decorator catches `mysql.connector.Error`, writes the stack trace to internal server files via `app.logger.exception`, and returns a generic client-safe error response:

```
{"error": "Database operation failed"}
```

This masks database structure details, table designs, and driver stack traces from attackers.

- **Path Traversal Prevention:** To ensure that file operations do not write or delete files outside of designated target folders, [app.py](#) checks the common path of the directories:

- `_analysis_directory` and `cleanup_audio_artifacts` resolve paths with `Path.resolve()` and verify that the base directory path matches the root uploads path using `os.path.commonpath()`. If a path goes outside, a `RuntimeError` is raised.

- **Audio Decompression & Safety Limits:** The audio processing service enforces limits on incoming audio before full loading or processing occurs. In [audio_analyzer.py](#), the `load_audio` function:

1. Verifies the file suffix against supported extensions.
2. Compares `path.stat().st_size` against `SafetyLimits.maximum_file_size_mb` (100MB max limit) before decoding starts.
3. Opens the file container with `soundfile.SoundFile` to inspect metadata, immediately raising an error if the declared audio duration exceeds `SafetyLimits.maximum_duration_seconds` (900s limit). This mitigates Zip-bomb or decompression-style attacks on memory.

- **Data Integrity & Schema Constraints:** In [schema.sql](#), strict constraints ensure data integrity:

- **UNIQUE** constraints on usernames and emails in the `user` table.
- Foreign keys on `refresh_token`, `revoked_access_token`, `audit_log`, `assessment`, and `registration_otp` tables mapping to `user(user_id)`.
- **ON DELETE CASCADE** ensures that deleting a user account cleans up all associated sensitive state records.

- **Audit Write Decoupling:** Audit log inserts are wrapped inside isolated try-except blocks in [app.py](#). A

failure in writing an audit record will be logged locally on the server but will not block the completion of a user request, preserving business data integrity.

3. Authentication and Authorization

All authentication routes are strictly rate-limited and perform validation checks before issuing JWT credentials.

Implementation Details

- **Double-Flow Registration:** A user cannot log in immediately after creating an account. Public signups insert an unverified record in `user` and write a hashed verification token into `registration_otp` ([app.py](#)). A 6-digit OTP code is emailed to the user.

- **Verification Attempts Capping:** During verification in [app.py](#), the OTP check is permitted only if `registration_otp.attempts < 5` and the code is not expired. Failed matches increment the count to prevent brute-forcing.

- **Constant-Time Verification:** Verification uses `secrets.compare_digest()` to compare code hashes in constant time to thwart timing attacks.

- **JWT Claims Validation:** On authorized routes, the decorator `@require_auth` in [app.py](#) decodes bearer tokens via PyJWT using HS256:

- Enforces claims: `sub` (User ID), `role`, `exp` (Expiration), `iat` (Issued At), `jti` (JWT Token ID).
- Ensures the `jti` has not been denylisted in the database.
- Ensures the corresponding user account is active (`is_active = TRUE`), role has not changed, and email is verified.

- **Credential Update Check:** Compares token issuance time `iat` against the database `security_updated_at` timestamp. If user details, password, or security settings updated after the token was issued, the token is rejected.

4. Password Hashing

Cleartext passwords are never stored or processed in log outputs. The application employs standard cryptographically strong hashing mechanisms.

Implementation Details

- **Argon2id Algorithm:** KaraOK uses Argon2id, configured via the `argon2-cffi` library in [app.py](#):

```
password_hasher = PasswordHasher(time_cost=2, memory_cost=19456, parallelism=1)
```

This configuration balances speed during user logins with high resistance to offline GPU/ASIC/FPGA-accelerated brute force attacks.

- **Salting:** Unique, randomized salts are automatically generated per password by the Argon2 library and stored as part of the encoded hash string in the `user.password` database field.
- **Automatic Re-hashing:** Upon successful logins, `app.py` evaluates the hash using `password_hasher.check_needs_rehash(user["password_hash"])`. If the configured Argon2 parameters have changed, the password is automatically rehashed and updated in the database.
- **Password Policy Enforcement:** Both frontend validation (`signup_screen.dart`) and backend validation (`app.py`) enforce password strength:
 - Minimum length: 12 characters; Maximum length: 128 characters.
 - Must contain uppercase, lowercase, numeric, and symbol characters.

5. Session Management

Sessions are managed using dual JWT access tokens and database-backed opaque refresh tokens.

Implementation Details

- **Token Lifetimes:** Access tokens are short-lived (15 minutes), mitigating the impact of token interception. Refresh tokens are long-lived (7 days).
- **Refresh Token Rotation (RTR):** When the client requests a new access token via `/api/auth/refresh` (`app.py`), the backend uses a row-level lock (`SELECT ... FOR UPDATE`) to verify the refresh token. Upon validation, the old refresh token is marked as immediately revoked (`revoked_at = UTC_TIMESTAMP()`), and a brand-new pair of access/refresh tokens is issued. This prevents concurrent token reuse and alerts the system to potential session hijacking if a token is reused.
- **Access Token Denylisting on Logout:** During logout (`app.py`), the backend extracts the `jti` of the presented access token. It inserts the `jti` and its expiration timestamp into the `revoked_access_token` table, preventing reuse of that access token before its natural expiry.
- **Storage:** On the client, when logout occurs or if a token refresh fails, the frontend runs `clearTokens()` in `api_service.dart` to remove keys from `FlutterSecureStorage` and clears the local session metadata.

6. Role-Based Access Control (RBAC)

The system restricts routes and administrative capabilities using a defined set of user roles.

Implementation Details

- **Defined Roles:** The roles are `owner`, `technician`, and `admin` (stored lowercased in the database).

- **Decorator Validation:** Endpoints are decorated with `@require_auth(*roles)` in [app.py](#). For example:

- `@require_auth("technician", "owner")` restricts audio uploads and tests to registered technician and owner accounts.

- `@require_auth("admin")` restricts listing users and reading centralized system audit logs.

- **Owner Ownership Checks:** On data retrieval and modification routes (such as fetching upload records or managing settings), routes execute SQL statements parameterized with the authenticated user ID:

```
WHERE a.user_id = %s
```

This prevents Insecure Direct Object Reference (IDOR) attacks, ensuring technicians or owners cannot access assessments, files, or settings belonging to another user.

- **Sign Up Restriction:** Public registration is restricted to `owner` or `technician` roles. Registering an `admin` user is rejected in the backend register endpoint ([app.py](#)) and must be done operationally on the database to prevent privilege escalation.

7. Input Validation and Sanitization

Input validation is enforced authoritatively on the server side, with secondary validations running on the client UI for interactive feedback.

Implementation Details

- **Text Cleaning & Whitespace Normalization:** The backend uses `clean_text()` to normalize whitespace (collapsing double spaces and trimming ends) and validates the length limits ([app.py](#)).

- **Numeric Range Bounds:** Input numbers (e.g., loudness, scores) are validated using `bounded_number()` to ensure they fall within acceptable ranges before processing, preventing buffer overflows or division-by-zero scenarios.

- **Email Validation:** Email inputs are normalized to lowercase and validated against the format regular expression `^[^\\s@]+@[^\\s@]+\\. [^\\s@]+$` in `clean_email()`.

- **SQL Injection Defenses:** The application uses parameterized queries through the MySQL connector library. Query values are passed as separate tuples (e.g. `cursor.execute("SELECT ... WHERE email = %s", (email,))`) rather than string-interpolated query constructs, ensuring inputs are treated strictly as data parameters rather than executable SQL commands.

- **Secure File Naming:** For uploaded audio files, [app.py](#) uses `secure_filename()` from `werkzeug.utils` to sanitize original filenames.

- **UUID Storing:** Rather than writing uploaded files using client-supplied names, files are written using cryptographically secure randomized UUIDs (`uuid.uuid4().hex`) and stored in user-specific folders, preventing file overwrites or execution of malicious upload payloads.

8. Forgot Password Flow

The forgot-password flow is designed to securely verify identities without exposing account credentials or leaking database attributes.

Implementation Details

- **Enumeration Protection:** The `/api/auth/forgot-password` endpoint ([app.py](#)) returns a uniform message regardless of whether the email address was found in the database:

```
{"message": "If the account exists, a temporary password has been sent."}
```

This prevents attackers from enumerating valid account emails.

- **CSPRNG Temporary Password:** If the user exists, a temporary password is generated using `secrets.choice()` and `secrets.SystemRandom().shuffle()` to guarantee randomness.

- **Immediate Session Invalidation:** Upon reset requests, the user's password record is updated with the temporary hash, and all existing active refresh sessions are immediately revoked in the database (`UPDATE refresh_token SET revoked_at = UTC_TIMESTAMP()`).

- **State Locking:** The field `requires_password_change` is set to `TRUE` for the user. When the user logs in using the temporary password, they are blocked by `@require_auth` from performing any API actions other than changing their password.

9. Audit Logging

Centralized audit trails and API logs are kept to track system state changes, authorization failures, and administrative actions.

Implementation Details

- **Central Audit Helper:** The `audit()` helper function in [app.py](#) inserts events into the `audit_log` table:

- Logs: `user_id` (the actor), `action`, `resource_type`, `resource_id`, `result` (success or failure), `ip_address`, `user_agent`, and brief description `details`.

- Logs are committed immediately.

- **Sensitive Data Exclusion:** Plaintext passwords, emails, verification codes, refresh/access tokens, and audio binaries are strictly excluded from all log records.

- **Request Logging:** The `@app.after_request` hook logs requests in `api_request_log`. It collects request methods, paths, endpoints, HTTP status codes, processing durations, IP addresses, and user agents. Query parameters, request body payloads, and authorization headers are ignored to maintain privacy.

- **Privilege Separation:** Access to both centralized audit logs (`/api/audit-logs`) and API request logs (`/api/api-requests`) requires authorization containing the `admin` role. In production, database permissions should restrict application connections from modifying or deleting existing records in `audit_log`.

10. Security Verification and Regression Tests

Automatic verification tests are kept inside the `backend/tests` folder. These verify security configuration behavior and prevent regressions.

- [test_security.py](#) contains test suites that check:
 - Sanitized request logs (`test_api_request_log_stores_sanitized_metadata`).
 - Password validation bounds (`test_password_policy_accepts_strong_password`, `test_password_policy_rejects_weak_password`).
 - Temporary password compliance (`test_temporary_password_meets_password_policy`).
 - Email normalization (`test_email_is_normalized`).
 - Deterministic hashing of secrets (`test_token_hash_is_deterministic_and_not_plaintext`).
 - Restriction of `admin` registration from public endpoints (`test_public_registration_rejects_privileged_role`).
- [test_audio_analyzer_safety.py](#) verifies:
 - That settings are loaded and checked correctly.
 - That oversized files are rejected before librosa attempts to decode them (`test_file_size_is_rejected_before_decode`), preventing memory exhaustion.
- [test_audio_validation.py](#) verifies:
 - Rejection of corrupted audio streams (`test_corrupted_audio_is_rejected`).
 - Correct cleanup of audio artifacts and prevention of traversal out of limits.